

The background features a grid of blue squares of varying sizes and opacities, creating a sense of depth. On the right side, there is a vertical column of binary digits (0s and 1s) in a light gray font.

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

Sectores

<<Atributos de instancia>>

t [] [Androide](#)

<<Constructor>>

Sectores (max:entero)

<<Comandos>>

asignar (a:[Androide](#), s: entero)

retirar (s:entero)

retirar (a:[Androide](#))

<<Consultas>>

cantSectores():entero

cantSectoresAsignados():entero

cantSectoresAndroide(a:[Androide](#)):entero

todosAsignados():boolean

estaAndroide(a:[Androide](#)):boolean

existeSector(s:entero):boolean

androideSector(s:entero):[Androide](#)

hayANave(n:NaveEspacial):boolean

clone():Sectores

Sectores (max:entero)

Crea una tabla para representar max sectores. Requiere max > 0

asignar (a:Androide, s:entero)

Asigna **a** al sector **s**. Requiere que el sector **s** sea válido y esté disponible.

retirar (s:entero)

Elimina la asignación del androide del sector **s**. Requiere que el sector **s** sea válido.

retirar (a:Androide)

Elimina todas las asignaciones del androide **a**.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

Sectores

<<Atributos de instancia>>

t [] [Androide](#)

<<Constructor>>

Sectores (max:entero)

<<Comandos>>

asignar (a:[Androide](#), s: entero)

retirar (s:entero)

retirar (a:[Androide](#))

<<Consultas>>

cantSectores():entero

cantSectoresAsignados():entero

cantSectoresAndroide(a:[Androide](#)):entero

todosAsignados():boolean

estaAndroide(a:[Androide](#)):boolean

existeSector(s:entero):boolean

androideSector(s:entero):[Androide](#)

hayANave(n:NaveEspacial):boolean

clone():Sectores

cantSectores():entero

Retorna el tamaño de la tabla

cantSectoresAsignados():entero

Retorna la cantidad de sectores que tienen asignado un androide.

cantSectoresAndroide(a:[Androide](#)):entero

Retorna la cantidad de sectores a las que está asignado el androide a.

todosAsignados ():boolean

Retorna true si todos los sectores tienen asignado un androide

estaAndroide(a:[Androide](#)):boolean

Retorna true si el androide a está asignado al menos a un sector.

Sectores

<<Atributos de instancia>>

t [] [Androide](#)

<<Constructor>>

Sectores (max:entero)

<<Comandos>>

asignar (a:[Androide](#), s: entero)

retirar (s:entero)

retirar (a:[Androide](#))

<<Consultas>>

cantSectores():entero

cantSectoresAsignados():entero

cantSectoresAndroide(a:Androide):entero

todosAsignados():boolean

estaAndroide(a:[Androide](#)):boolean

existeSector(s:entero):boolean

androideSector(s:entero):[Androide](#)

hayANave(n:NaveEspacial):boolean

clone():Sectores

existeSector(s:entero):boolean

Retorna true si s es un sector válido

androideSector (s:entero):Androide

Retorna al androide asignado al sector s
o nulo

hayANave(n:NaveEspacial):boolean

Retorna verdadero si hay al menos un
androide asignado a la nave n.

clone():Sectores

Implementa clone en profundidad

Sectores
<<Atributos de instancia>> t [] Androide
<<Constructor>> Sectores (max:entero) <<Comandos>> asignar (a: Androide , s: entero) retirar (s:entero) retirar (a: Androide) <<Consultas>> cantSectores():entero cantSectoresAsignados():entero cantSectoresAndroide(a:Androide):entero todosAsignados():boolean estaAndroide(a: Androide):boolean existeSector(s:entero):boolean androideSector(s:entero): Androide hayANave(n:NaveEspacial):boolean clone():Sectores

Androide
<<Atributos de instancia>> vocablos: entero nave: NaveEspacial
<<Constructor>> Androide (n: NaveEspacial) <<Comandos>> establecerNave(n: NaveEspacial) aprenderVocablos(v: entero) copy(a: Androide) <<Consultas>> obtenerNave(): NaveEspacial obtenerVocablos(): entero clone(): Androide equals(a: Androide): boolean

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

```
class Sectores {  
    private Androide[] t;  
    //Constructor  
    public Sectores(int max) {  
        /*Crea una tabla para representar max sectores*/  
        t = new Androide [max];  
    }  
}
```


//Comandos

```
public void asignar(Android a, int s) {  
    /*Asigna a al sector s. Requiere que el  
    sector s sea válido y esté disponible*/  
    t[s] = a;  
}
```

1 1 0 0
0 0 1 1
0 1 1 0
1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

```
public void retirar(int s) {  
    /*Elimina la asignación del androide del sector  
    s. Requiere que el sector s sea válido.*/  
    t[s] = null;  
}
```

Si el sector no tenía un androide asignado no se produce ningún cambio.


```
public void retirar (Androide a) {  
    /*Elimina todas las asignaciones del androide a.*/  
    for (int i=0;i < cantSectores();i++)  
        if (t[i] == a)  
            t[i] = null;  
}
```

Aunque el diseñador no lo especifica explícitamente asumimos que la comparación es por identidad.

```
//Consultas
public int cantSectores () {
    //Retorna el tamaño de la tabla
    return t.length;
}
public int cantSectoresAsignados () {
    /*Retorna la cantidad de sectores que tienen
    asignado un androide*/
    int cant = 0;
    for (int i=0; i < cantSectores() ; i++)
        if (t[i] != null) cant++;
    return cant;
}
```

```
public int cantSectoresAndroide(Androide a) {  
    /*Retorna la cantidad de sectores que tienen  
    asignado al androide a.*/  
    int cant=0;  
    for (int i=0;i < cantSectores();i++)  
        if (t[i] == a)  
            cant++;  
    return cant;  
}
```

```
public int todosAsignados() {  
    /*Retorna true si todos los sectores tienen  
    asignado un androide.*/  
    boolean todos=true;  
    for (int i=0;i < cantSectores() && todos;i++)  
        todos = t[i]!=null;  
    return todos;  
}
```

```
public boolean estaAndroide(Android a) {  
    /*Retorna true si el androide a está asignado al  
    menos a un sector.*/  
    boolean esta=false;  
    for (int i=0;i < cantSectores() && !esta;i++)  
        esta = t[i] == a;  
    return esta;  
}
```

Si el parámetro no está ligado retorna true si hay un sector sin asignar

```
public boolean existeSector(int s) {  
    /*Retorna true si s es un sector válido*/  
    return 0<=s && s<cantSectores();  
}
```

```
public Androide androideSector(int s) {  
    /*Retorna al androide asignado al sector s  
    o nulo*/  
    Androide a=null;  
    if (existeSector(s))  
        a = t[s];  
    return a;  
}
```

```
public boolean hayANave(NaveEspacial n) {  
    /*Retorna verdadero si hay al menos un  
    androide asignado a la nave n.*/  
    boolean esta=false;  
    for (int i=0;i < cantSectores() && !esta;i++)  
        esta = (t[i] != null && t[i].obtenerNave()==n) ;  
    return esta;  
}
```



El contrato de la clase **Androide** establece que todo androide está asociado a una nave.


```
public Sectores clone() {  
    /*Implementa clone en profundidad*/  
    Sectores nueva = new Sectores(cantSectores());  
    for (int i=0;i < cantSectores();i++)  
        if (t[i] != null)  
            nueva.asignar(t[i].clone(),i);  
    return nueva;  
}
```



`t[i]` puede recibir cualquiera de los servicios que brinda la clase **Androide**

*Una compañía de transporte tiene un **estacionamiento** formado por un conjunto de **unidades de estacionamiento** en el que se ubican los **micros**. En un momento dado una unidad puede estar ocupada por un micro o libre. Las unidades libres y ocupadas están intercaladas. En una unidad de estacionamiento claramente solo puede estar estacionado un micro y un mismo micro solo puede estar estacionado en una unidad.*



0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

El estacionamiento puede ser modelado por un arreglo en el cual el subíndice indica el número de unidad. Inicialmente el estacionamiento está vacío.

*Cuando un micro **estaciona** en una unidad **se asigna un objeto a un elemento arreglo**.*

*Cuando se **retira** un micro se asigna nulo a un elemento del arreglo.*



0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

Estacionamiento
<<Atributos de instancia>> t [] Micro
<<Constructor>> Estacionamiento (max:entero)
<<Comandos>> estacionar(unMicro: Micro ,p:entero) estacionar(unMicro: Micro) retirar(p:entero) retirar(unMicro: Micro) inspeccion() <<Consultas>> cantUnidades():entero cantUnidadesOcupadas():entero todasOcupadas():boolean estaMicro(unMicro: Micro):boolean existeUnidad(p:entero):boolean microUnidad(p:entero): Micro cantVerificación(m:entero):entero cantDisponibles():entero

Estacionamiento (max:entero) Crea una tabla para representar max unidades de estacionamiento. Requiere max >0
estacionar (unMicro:Micro) Busca la primera unidad libre, si existe asigna el micro a esa unidad. Requiere que unMicro no esté estacionado en una unidad
estacionar (unMicro:Micro,p:entero) Estaciona unMicro en la unidad p. Requiere la unidad controlada y vacía.
retirar (p:entero) Elimina el micro de la unidad p. Requiere controlada la unidad.
retirar (unMicro:Micro) Elimina el micro con la misma patente que unMicro. Requiere unMicro ligado.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

Estacionamiento
<<Atributos de instancia>> t [] Micro
<<Constructor>> Estacionamiento (max:entero) <<Comandos>> estacionar(unMicro: Micro ,p:entero) estacionar(unMicro: Micro) retirar(p:entero) retirar(unMicro: Micro) inspeccion() <<Consultas>> cantUnidades():entero cantUnidadesOcupadas():entero todasOcupadas():boolean estaMicro(unMicro: Micro):boolean existeUnidad(p:entero):boolean microUnidad(p:entero): Micro cantVerificación(m:entero):entero cantDisponibles():entero

estaMicro(unMicro:Micro):boolean Retorna true si el micro con la patente de unMicro está asignado a alguna unidad de Estacionamiento. Requiere unMicro ligado.
existeUnidad(p:entero):boolean Retorna true si p es una unidad de estacionamiento válida.
microUnidad (p:entero):Micro Retorna el micro estacionado en la unidad p. Requiere p válida.
cantVerificación(m:entero):entero Retorna cuántos micros hicieron su última verificación en el mes m. Requiere m válido.
cantDisponibles():entero Retorna cuántos micros tienen al menos un asiento disponible.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

Estacionamiento
<<Atributos de instancia>> t [] Micro
<<Constructor>> Estacionamiento (max:entero)
<<Comandos>> estacionar(unMicro: Micro ,p:entero) estacionar(unMicro: Micro) retirar(p:entero) retirar(unMicro: Micro) inspeccion()
<<Consultas>> cantUnidades():entero cantUnidadesOcupadas():entero todasOcupadas():boolean estaMicro(unMicro: Micro):boolean existeUnidad(p:entero):boolean microUnidad(p:entero): Micro cantVerificación(m:entero):entero cantDisponibles():entero

Micro
<<Atributos de instancia>> patente:String fechaUltVTV:Fecha cantDisponibles:entero
<<Constructor>> Micro(p:String,f:Fecha)
<<Comandos>> ocupar(a:entero) liberar(a:entero) establecerFechaUltVTV(f:Fecha)
<<Consulta>> obtenerPatente():String obtenerFechaUltVTV():Fecha obtenerCantDisponibles():entero estaOcupado(a:entero):boolean igualPatente(unMicro:Micro):boolean
La patente y la fecha de la última verificación siempre están ligadas

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1

Estacionamiento
<<Atributos de instancia>> t [] Micro
<<Constructor>> Estacionamiento (max:entero) <<Comandos>> estacionar(unMicro: Micro ,p:entero) estacionar(unMicro: Micro) retirar(p:entero) retirar(unMicro: Micro) inspeccion() <<Consultas>> cantUnidades():entero cantUnidadesOcupadas():entero todasOcupadas():boolean estaMicro(unMicro: Micro):boolean existeUnidad(p:entero):boolean microUnidad(p:entero): Micro cantVerificación(m:entero):entero cantDisponibles():entero

Micro
<<Atributos de instancia>> patente:String fechaUltVTV:Fecha asientos[] boolean
<<Constructor>> Micro(p:String) <<Comandos>> ocupar(a:entero) liberar(a:entero) establecerFechaUltVTV(f:Fecha) <<Consulta>> obtenerPatente():String obtenerFechaUltVTV():Fecha obtenerCantDisponibles():entero estaOcupado(a:entero):boolean igualPatente(unMicro:Micro):boolean

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1


```
class Estacionamiento {  
    private Micro[] t;  
    //Constructor  
    public Estacionamiento(int max) {  
        /*Crea una tabla para representar max unidades  
        de estacionamiento. Requiere max>0 */  
        t= new Micro [max];  
    }  
}
```



```
//Comandos
```

```
public void estacionar (Micro unMicro) {
/*Busca la primera unidad libre, si existe
asigna el micro a esa unidad. Requiere que unMicro
no esté estacionado en otra unidad*/
    int i = 0;
    while (i < cantUnidades() && t[i] != null)
        i++;
    if (i < cantUnidades())
        t[i] = unMicro;
}
```

```
//Comandos
```

```
public void estacionar (Micro unMicro) {
/*Busca la primera unidad libre y asigna unMicro a
la unidad. Requiere que unMicro no esté estacionado
en otra unidad y que haya una unidad libre*/
    int i = 0;
    while (t[i] != null)
        i++;
    t[i] = unMicro;
}
```

El comentario indica que es la clase cliente la responsable de garantizar que hay un unidad libre, por ejemplo invocando a **todasOcupadas ()**

```
public void estacionar(Micro unMicro,int p) {  
    /*Estaciona unMicro en la unidad p. Requiere que la  
    unidad p sea válida y esté disponible*/  
    if(!estaMicro(unMicro))  
        t[p] = unMicro;  
}
```

Si la unidad tenía un micro asignado se produce un **error de aplicación**.

```
public void retirar(int p) {  
    /*Elimina el micro de la unidad p. Requiere  
    controlada la unidad*/  
    t[p] = null;  
}
```

Si la unidad no tenía un micro estacionado no se produce ningún cambio.

```
public void retirar(Micro unMicro) {  
    /*Elimina el micro con la misma patente que  
    unMicro, requiere unMicro ligado */  
    int i = 0; boolean encontro=false;  
    while (i < cantUnidades() &&!encontro) {  
        if (t[i] != null &&  
            t[i].igualPatente(unMicro)); {  
            encontro = true;  
            t[i] = null;  
        }  
        i++;  
    }  
}
```

```
//Consultas  
public int cantUnidades () {  
    //Retorna el tamaño de la tabla  
    return t.length;  
}
```

La funcionalidad es análoga a la consulta cantSectores() de la clase **Sectores**.

```
public int cantUnidadesOcupadas () {  
    /*Retorna la cantidad de unidades ocupadas por un  
    micro*/  
    int i = 0; int cant = 0;  
    while (i < cantUnidades ()) {  
        if (t[i] != null)  
            cant++;  
        i++;  
    }  
    return cant;  
}
```

La funcionalidad es análoga a la consulta cantSectoresAsignados() de la clase **Sectores**.

```
public boolean estaMicro(Micro unMicro) {
    /*Retorna true si el micro con la patente de
    unMicro está asignado a alguna unidad de
    Estacionamiento. Requiere unMicro ligado.*/
    boolean esta=false;
    String p = unMicro.obtenerPatente();
    for (int i=0;i < cantUnidades() && !esta;i++)
        esta = t[i] != null &&
                t[i].obtenerPatente().equals(p);
    return esta;
}
```

La funcionalidad es similar a `estaAndroide` de la clase `Sectores`.



```
public boolean existeUnidad (int p) {  
    return p >= 0 & p < t.length;  
}
```

```
public Micro microUnidad (int p) {  
    /*Retorna el micro estacionado en la unidad p.  
    Requiere p válida */  
    return t[p];  
}
```

1 1 0 0
0 0 1 1
0 1 1 0
1 1 1 0
1 1 0 0
0 0 1 1
0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0

La responsabilidad de `microUnidad` es diferente a `androideSector` de la clase `Androide`.

```
public int cantVerificacion (int m){  
    /*Retorna cuántos micros hicieron su última  
    verificación en el mes m. Requiere m válido.*/  
    int cant = 0;  
    for (int i=0;i < cantUnidades();i++)  
        if (t[i]!=null &&  
            t[i].obtenerFechaUltVTV().obtenerMes()==m)  
            cant++;  
    return cant;  
}
```

```
public int cantDisponibles () {  
    /*Retorna cuántos micros tienen al menos  
    un asiento disponible*/  
    int cant = 0;  
    for (int i=0; i < cantUnidades() ; i++)  
        if (t[i] != null &&  
            t[i].obtenerCantDisponibles() > 0)  
            cant++;  
    return cant;  
}
```



Si observamos el comportamiento de la clase **Sectores** definida antes y la clase **Estacionamiento** definida ahora, podemos notar un *patrón* en la representación de los datos y en las operaciones.

Los sectores del videojuego y las unidades del estacionamiento se asocian a elementos de un arreglo.

En ambas clases el constructor crea un arreglo.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1





```
class Sectores {  
    private Androide[] t;  
    //Constructor  
    public Sectores(int max) {  
        /*Crea una tabla para representar max sectores */  
        t= new Androide [max];  
    }  
}
```

```
class Estacionamiento {  
    private Micro[] t;  
    //Constructor  
    public Estacionamiento(int max) {  
        /*Crea una tabla para representar max unidades de  
        estacionamiento */  
        t= new Micro [max];  
    }  
}
```

1 1 0 0
0 0 1 1
1 0 1 1 0
1 1 1 0
1 1 0 0
0 0 1 1
0 1 1 0
1 1 1 0
0 0 1
1 1
0



//Comandos

```
public void asignar(Android a, int s) {  
    /*Asigna a al sector s. Requiere que el  
    sector s sea válido y esté disponible*/  
    t[s] = a;  
}
```

```
public void estacionar(Micro unMicro, int p) {  
    /*Estaciona unMicro en la unidad p. Requiere que la  
    unidad p sea válida y esté disponible. Requiere que  
    unMicro no esté estacionado*/  
    t[p] = unMicro;  
}
```



La consulta que decide si un androide está asignado a algún sector es **similar** a la consulta que decide si un micro está estacionado en alguna unidad, aunque la búsqueda en el primer caso es por identidad y en el segundo por número de patente.

En la consulta que retorna el micro estacionado en una unidad la responsabilidad de controlar que el sector sea válido es de la clase cliente, en la consulta que retorna el androide asignado a un sector, la responsabilidad es de la clase Sectores.

Además, un micro solo puede ocupar una unidad mientras que el mismo androide puede estar asignado a varios sectores.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1



Reusabilidad

Sectores

<<Atributos de instancia>>

t [] **Androide**

<<Constructor>>

Sectores (max:entero)

<<Comandos>>

asignar(a:**Androide**, s:entero)

retirar (s:entero)

retirar (a:**Androide**)

<<Consultas>>

cantSectores():entero

cantSectoresAsignados():entero

cantSectoresAndroide(a:Androide):entero

todosAsignados ():boolean

estaAndroide(a:**Androide**):boolean

existeSector(s:entero):boolean

androideSector (s:entero):**Androide**

hayANave(n:NaveEspacial):boolean

clone():Sectores

Estacionamiento

<<Atributos de instancia>>

t [] **Micro**

<<Constructor>>

Estacionamiento (max:entero)

<<Comandos>>

estacionar(unMicro:**Micro**,p:entero)

estacionar(unMicro:**Micro**)

retirar (p:entero)

retirar (unMicro:**Micro**)

inspeccion()

<<Consultas>>

cantUnidades ():entero

cantUnidadesOcupadas():entero

todasOcupadas ():boolean

estaMicro(unMicro:**Micro**):boolean

existeUnidad(p:entero):boolean

microUnidad (p:entero):**Micro**

cantVerificación(m:entero):entero

cantDisponibles():entero



Algunos servicios son específicos de **Sectores**, otros de **Estacionamiento**.

Adoptamos la convención de llamar **tabla** a una estructura que encapsula un arreglo que mantiene referencias nulas y ligadas intercaladas y brinda operaciones para insertar, eliminar, buscar y procesar de alguna manera los elementos.

0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
0 1 1 0 0
1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1



Reusabilidad

Sectores

<<Atributos de instancia>>

t [] **Androide**

<<Constructor>>

Sectores (max:entero)

<<Comandos>>

asignar(a:**Androide**, s:entero)

retirar (s:entero)

retirar (a:**Androide**)

<<Consultas>>

cantSectores():entero

cantSectoresAsignados():entero

cantSectoresAndroide(a:Androide):entero

todosAsignados ():boolean

estaAndroide(a:**Androide**):boolean

existeSector(s:entero):boolean

androideSector (s:entero):**Androide**

hayANave(n:NaveEspacial):boolean

clone():Sectores

Estacionamiento

<<Atributos de instancia>>

t [] **Micro**

<<Constructor>>

Estacionamiento (max:entero)

<<Comandos>>

estacionar(unMicro:**Micro**,p:entero)

estacionar(unMicro:**Micro**)

retirar (p:entero)

retirar (unMicro:**Micro**)

inspeccion()

<<Consultas>>

cantUnidades ():entero

cantUnidadesOcupadas():entero

todasOcupadas ():boolean

estaMicro(unMicro:**Micro**):boolean

existeUnidad(p:entero):boolean

microUnidad (p:entero):**Micro**

cantVerificación(m:entero):entero

cantDisponibles():entero